

Aufgabenstellung: Bibliotheken in Java am Beispiel eines Veranstaltungs- und Ticketsystems

(Java Standard Library, Collections Framework und Datenverarbeitung)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das zentrale Bibliotheken der Java-Plattform in einem neuen Anwendungskontext gezielt einsetzt.

Sie lernen dabei,

- wie die **Java Standard Library** in einem praktischen Verwaltungssystem verwendet wird,
- wie Klassen aus `java.lang`, `java.util`, `java.time` und `java.math` sinnvoll kombiniert werden,
- wie das **Java Collections Framework (JCF)** zur Verwaltung von Veranstaltungen und Tickets genutzt wird,
- wie Daten gespeichert, durchsucht, gruppiert, sortiert und ausgewertet werden,
- wie Bibliotheken dazu beitragen, Programme klarer, robuster und besser erweiterbar zu gestalten.

Der Fokus liegt auf dem Verständnis, dass vorhandene Bibliotheken nicht nur Hilfsmittel sind, sondern ein zentraler Bestandteil guter Softwareentwicklung.

Fachlicher Hintergrund

Viele Anwendungen verwalten strukturierte Daten, zum Beispiel:

- Veranstaltungen
- Termine
- Preise
- Kategorien
- Tickets
- Anmeldungen

Java stellt dafür bereits viele geeignete Klassen bereit.

Dazu gehören unter anderem:

- `java.lang` für grundlegende Klassen wie `String`, `Math`, `Integer`

- `java.util` für Collections, Listen, Mengen, Maps und Sortiermechanismen
- `java.time` für Datums- und Zeitangaben wie Veranstaltungsdaten
- `java.math` für genaue Preisberechnungen mit `BigDecimal`

Besonders wichtig ist wieder das **Java Collections Framework (JCF)**.
Es hilft dabei, viele Objekte gleichzeitig zu verwalten und systematisch auszuwerten.

In dieser Aufgabe geht es um ein kleines Veranstaltungs- und Ticketsystem:

- Veranstaltungen werden angelegt,
- Veranstaltungen besitzen Namen, Kategorien, Preise und Termine,
- mehrere Veranstaltungen werden verwaltet,
- Kategorien sollen eindeutig erfasst werden,
- Veranstaltungen sollen sortiert und nach Eigenschaften gefiltert werden,
- Preise und Einnahmen sollen berechnet werden.

Damit eignet sich die Aufgabe gut zur Übung der Standardbibliotheken in einem neuen fachlichen Zusammenhang.

Vorgegebene Klassenstruktur

Im Package `bibliotheken.event` sollen folgende Klassen entwickelt und verwendet werden:

- `Veranstaltung`
- `VeranstaltungsVerwaltung`
- `EventAnalyse`
- `Main`

Zusätzlich sollen geeignete Klassen aus den Bibliotheken verwendet werden:

- `String`, `Math`, `Integer` aus `java.lang`
- `List`, `ArrayList`, `Set`, `HashSet`, `Map`, `HashMap`, `Collections`, `Comparator` aus `java.util`
- `LocalDate`, `Period`, `DateTimeFormatter` aus `java.time`
- `BigDecimal` aus `java.math`

Aufgabe 1: Analyse der Java Standard Library

Ziel

Sie untersuchen erneut die Rolle der Java Standard Library, diesmal bezogen auf ein Veranstaltungs- und Ticketsystem.

Anforderungen

Funktionale Anforderungen

Erklären Sie,

- was die Java Standard Library ist,
- warum sie bei einem Verwaltungssystem hilfreich ist,
- welche Aufgaben die Pakete `java.lang`, `java.util`, `java.time` und `java.math` im Beispiel übernehmen können.

Technische Anforderungen

Beziehen Sie sich auf mindestens folgende Klassen:

- `String`
- `Math`
- `ArrayList`
- `HashMap`
- `LocalDate`

- `BigDecimal`

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass Standardbibliotheken typische Probleme direkt unterstützen,
- dass Bibliotheken auch in neuen fachlichen Kontexten sinnvoll eingesetzt werden können,
- dass dieselben Pakete in vielen unterschiedlichen Anwendungen eine Rolle spielen.

Aufgabe 2: Verwendung von `java.lang`

Ziel

Sie verwenden grundlegende Klassen aus `java.lang` im Zusammenhang mit Veranstaltungsdaten.

Zu entwickelnde Inhalte

Nutzen Sie mindestens:

- `String`
 - `Math`
 - Wrapper-Klassen wie `Integer` oder `Double`
-

Anforderungen

Funktionale Anforderungen

Entwickeln Sie Code, so dass:

- Veranstaltungsnamen als Text verarbeitet werden können,
- Texte verändert oder ausgewertet werden können,
- eingegebene Zahlen aus Zeichenketten umgewandelt werden können,
- einfache mathematische Operationen durchgeführt werden können.

Technische Anforderungen

Verwenden Sie zum Beispiel:

- `length()`

- `toUpperCase()`
 - `substring()`
 - `Math.round()`
 - `Math.sqrt()`
 - `Integer.parseInt()`
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie `java.lang` in typischen Programmen genutzt wird,
 - wie Texte und Zahlen in Verwaltungsprogrammen verarbeitet werden,
 - warum Grundklassen für fast jedes Java-Programm wichtig sind.
-

Aufgabe 3: Entwicklung der Klasse Veranstaltung

Ziel

Sie entwickeln eine Fachklasse, die eine Veranstaltung beschreibt.

Zu entwickelnde Klassenstruktur

Die Klasse `Veranstaltung` soll mindestens enthalten:

- `titel : String`
- `preis : BigDecimal`
- `kategorie : String`
- `datum : LocalDate`

Beispielhafte Methoden:

- Konstruktor
- Getter
- `beschreibe()`

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse so, dass:

- eine Veranstaltung fachlich beschrieben werden kann,
- Preisangaben präzise gespeichert werden,
- ein Veranstaltungsdatum gespeichert werden kann.

Technische Anforderungen

Verwenden Sie:

- String
- BigDecimal
- LocalDate

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Bibliotheken direkt in Fachklassen eingebunden werden,
- wie Preis- und Datumsangaben mit geeigneten Klassen modelliert werden,
- wie ein eigenes Objekt aus mehreren Standardbibliothekstypen aufgebaut werden kann.

Aufgabe 4: Verwendung von java.time

Ziel

Sie arbeiten mit Datumswerten im Kontext von Veranstaltungen.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie Code, so dass:

- Veranstaltungsdaten gespeichert werden,
- Datumsangaben formatiert ausgegeben werden,
- berechnet werden kann, wie lange es noch bis zu einer Veranstaltung dauert.

Technische Anforderungen

Verwenden Sie:

- `LocalDate.now()`
- `LocalDate.of(...)`

- `Period.between(...)`
 - `DateTimeFormatter`
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Datumswerte für Termine und Fristen modelliert werden,
 - wie zukünftige Veranstaltungen ausgewertet werden,
 - wie Datumsangaben benutzerfreundlich dargestellt werden.
-

Aufgabe 5: Verwendung von `java.math.BigDecimal`

Ziel

Sie analysieren und nutzen `BigDecimal` für präzise Preisberechnungen.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie Code, so dass:

- Ticketpreise exakt gespeichert werden,
- Gesamteinnahmen berechnet werden,
- Rabatte oder Zuschläge präzise berücksichtigt werden können.

Technische Anforderungen

Verwenden Sie:

- `BigDecimal`
- `add(...)`
- `subtract(...)`
- `multiply(...)`
- `optional divide(...)`

Ziel dieser Aufgabe

Sie sollen verstehen,

- warum `BigDecimal` für Preise und Einnahmen besser geeignet ist als `double`,
 - wie kaufmännische Berechnungen in Java korrekt umgesetzt werden,
 - wie Methoden statt Operatoren verwendet werden.
-

Aufgabe 6: Einführung in das Java Collections Framework

Ziel

Sie untersuchen die Rolle des JCF in einem Veranstaltungsverwaltungssystem.

Anforderungen

Funktionale Anforderungen

Erklären Sie,

- was das JCF ist,
- warum ein Verwaltungssystem passende Datenstrukturen braucht,

- worin sich List, Set und Map unterscheiden.

Technische Anforderungen

Beziehen Sie sich auf:

- List
- Set
- Map
- ArrayList
- HashSet
- HashMap

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass Datenverwaltung ohne geeignete Collections schnell unübersichtlich wird,
- dass unterschiedliche Strukturen unterschiedliche Zwecke erfüllen,
- dass die Wahl der Datenstruktur die Lösungsqualität beeinflusst.

Aufgabe 7: Entwicklung der Klasse VeranstaltungsVerwaltung

Ziel

Sie entwickeln eine verwaltende Klasse, die mehrere Veranstaltungen speichert.

Zu entwickelnde Klassenstruktur

Die Klasse `VeranstaltungsVerwaltung` kann mindestens enthalten:

- `private List<Veranstaltung> veranstaltungen;`

Beispielhafte Methoden:

- `veranstaltungHinzufuegen(Veranstaltung veranstaltung)`
- `alleVeranstaltungenAusgeben()`
- `veranstaltungenNachKategorie(String kategorie)`
- `gesamtpreisBerechnen()`

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse so, dass:

- mehrere Veranstaltungen gespeichert werden können,
- Veranstaltungen nach Kategorie gefiltert werden können,
- Summen über Preise berechnet werden können,
- mit mehreren Objekten systematisch gearbeitet wird.

Technische Anforderungen

Verwenden Sie:

- `List<Veranstaltung>`
- `ArrayList`
- Schleifen oder `for-each`
- `BigDecimal` für Summen

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Fachobjekte in Listen verwaltet werden,
- wie Daten aus Sammlungen gelesen und verarbeitet werden,
- wie verwaltende Klassen mit Collections arbeiten.

Aufgabe 8: Analyse von Set und Map

Ziel

Sie erweitern das System um eindeutige Kategorien und Zuordnungen.

Anforderungen

Funktionale Anforderungen

Entwickeln oder analysieren Sie Lösungen so, dass:

- doppelte Kategorien vermieden werden können,
- Veranstaltungen nach Titel oder Kategorie zugeordnet werden können,

- Schlüssel-Wert-Beziehungen sichtbar werden.

Technische Anforderungen

Verwenden Sie:

- `Set<String>`
- `HashSet`
- `Map<String, Veranstaltung>` oder `Map<String, List<Veranstaltung>>`
- `HashMap`

Ziel dieser Aufgabe

Sie sollen verstehen,

- wann ein `Set` geeigneter ist als eine `List`,
- wie `Map`-Strukturen gezielte Zuordnungen ermöglichen,
- wie unterschiedliche `Collections` unterschiedliche Anforderungen erfüllen.

Aufgabe 9: Sortieren und Vergleichen von Daten

Ziel

Sie sortieren Veranstaltungen mithilfe von Bibliotheksfunktionen.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie Code, so dass:

- Veranstaltungen alphabetisch nach Titel sortiert werden können,
- Veranstaltungen nach Preis sortiert werden können,
- Veranstaltungen nach Datum sortiert werden können.

Technische Anforderungen

Verwenden Sie:

- `Collections.sort(...)` oder `List.sort(...)`
- `Comparator`
- Lambda-Ausdrücke optional

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Bibliotheken flexible Sortierung ermöglichen,
- wie Vergleichskriterien formuliert werden,
- wie Daten je nach Ziel unterschiedlich geordnet werden können.

Aufgabe 10: Bibliotheken zur Verarbeitung von Daten

Ziel

Sie analysieren das Zusammenspiel mehrerer Bibliotheken im Veranstaltungssystem.

Anforderungen

Funktionale Anforderungen

Erklären und entwickeln Sie Beispiele dazu, wie

- Texte verarbeitet werden,
- Preise berechnet werden,
- Datumsangaben ausgewertet werden,
- Veranstaltungen in Collections verwaltet werden,
- Daten gefiltert, gruppiert und sortiert werden.

Technische Anforderungen

Beziehen Sie sich auf:

- `java.lang`
- `java.util`
- `java.time`
- `java.math`
- JCF-Strukturen wie `List`, `Set`, `Map`

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass Bibliotheken im Zusammenspiel verwendet werden,

- dass Datenverarbeitung meist mehrere Pakete gleichzeitig benötigt,
 - dass die passende Bibliothekswahl die Qualität eines Programms verbessert.
-

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was die Java Standard Library ist,
 - wichtige Pakete der Standardbibliothek benennen und einordnen,
 - Klassen aus `java.lang`, `java.util`, `java.time` und `java.math` anwenden,
 - den Unterschied zwischen `List`, `Set` und `Map` erklären,
 - mit `ArrayList`, `HashSet` und `HashMap` arbeiten,
 - Datumsangaben mit `LocalDate` und `DateTimeFormatter` verarbeiten,
 - Geldbeträge mit `BigDecimal` berechnen,
 - Objekte in `Collections` speichern, filtern und sortieren,
 - Bibliotheken gezielt in einem Fachkontext einsetzen.
-

Didaktische Einordnung

Diese Übungsaufgabe überträgt bekannte Bibliothekskonzepte auf einen neuen fachlichen Zusammenhang.

Sie knüpft an folgende Themen an:

- Klassen und Objekte
- Attribute und Methoden
- Standardbibliotheken
- Strings und Zahlenverarbeitung
- Datumsverarbeitung
- präzise Berechnungen
- Collections
- Listen, Mengen und Maps
- Sortieren und Vergleichen
- strukturierte Datenverarbeitung

Die Aufgabe eignet sich gut, um vorhandenes Wissen nicht nur zu wiederholen, sondern in einer leicht veränderten Problemsituation aktiv anzuwenden.

Reflexions- und Kontrollfragen

Kontrollfragen zur Java Standard Library

1. Welche Rolle spielt die Java Standard Library in dieser Aufgabe?
2. Welche Pakete werden besonders häufig verwendet?
3. Warum muss nicht jede Funktion selbst programmiert werden?

Kontrollfragen zu java.lang

4. Wie kann ein Veranstaltungstitel mit `String` verarbeitet werden?
5. Wofür könnte `Math` in einem Ticketsystem genutzt werden?
6. Warum ist `Integer.parseInt(...)` nützlich?

Kontrollfragen zu java.time

7. Warum ist `LocalDate` für Veranstaltungsdaten geeignet?
8. Wie kann ein Datum formatiert ausgegeben werden?
9. Wie kann berechnet werden, wie lange es noch bis zu einer Veranstaltung dauert?

Kontrollfragen zu java.math

10. Warum sollte ein Ticketpreis nicht einfach als `double` gespeichert werden?

11. Wie werden Preisänderungen mit `BigDecimal` berechnet?

Kontrollfragen zum JCF

12. Warum ist `List<Veranstaltung>` für die Verwaltung sinnvoll?

13. Wann ist ein `Set<String>` für Kategorien hilfreich?

14. Wozu könnte eine `Map<String, Veranstaltung>` verwendet werden?

15. Worin unterscheiden sich `ArrayList`, `HashSet` und `HashMap`?

16. Warum ist die Wahl der Collection wichtig?

Kontrollfragen zur Datenverarbeitung

17. Wie können Veranstaltungen gefiltert werden?

18. Wie können Kategorien eindeutig gesammelt werden?

19. Wie können Veranstaltungen sortiert werden?

20. Warum ist das Zusammenspiel mehrerer Bibliotheken typisch für Java-Programme?
